

SnapStak Mobile

Complete Engineering Reference

.NET MAUI Android app for deconstructing mobile WebView applications

A full code breakdown for a developer joining the project: how to open and build it in Visual Studio 2022, how the on-device capture works, and how every service and file fits.

ConteX Law applied to mobile WebView deconstruction, running fully on-device.

Russel Hawkins | SnapStak.ai

Licensed under AGPL-3.0 | github.com/SnapStak-AI/SnapStak-Mobile

Contents

1. Introduction.....	4
1.1 What this software does.....	4
1.2 Why it is built this way: ConteX Law	4
1.3 How Mobile differs from the Desktop engine	4
1.4 Who this document is for.....	5
2. The Visual Studio 2022 Solution	6
2.1 Prerequisites	6
2.2 Solution structure	6
2.3 The shared engine and how it is linked	6
2.4 NuGet packages	7
2.5 Build configurations: Standalone vs Enterprise.....	7
2.6 Running it on a device or emulator	8
3. Architecture Overview.....	9
3.1 The end-to-end flow	9
3.2 The four pillars on mobile.....	9
3.3 Where the data lives	9
4. App Discovery: Finding WebView Apps.....	11
4.1 AppScannerService - the two-layer APK scan.....	11
4.2 Signatures and SignatureCacheService	11
4.3 AssetExtractorService - unpacking local apps.....	12
5. The Capture Layer (Android WebView).....	13
5.1 LocalAssetWebViewHandler - the heart of capture	13
5.2 Local-asset apps vs remote-URL apps.....	13
5.3 Defeating CSP and framework JavaScript.....	14
5.4 Injecting the engine and the native bridge stubs.....	14
5.5 content.mobile.js - the extractor	14
6. The On-Device Pipeline.....	16
6.1 DeconstructPage - orchestrating extraction.....	16
6.2 MobileConteXPipelineService - building the pillars.....	16
6.3 The translator plugins.....	16
6.4 FilePillarStorage - on-device storage.....	17
7. Code Generation and Export.....	18
7.1 TransformPage - framework choice and generation.....	18
7.2 The Standalone and Enterprise generation paths.....	18
7.3 Exporting plugin outputs via the share sheet.....	18
8. Services and Supporting Systems.....	19
8.1 LocalEngineService	19
8.2 PairingService and subscription.....	19
8.3 AppDatabaseService	19
8.4 MobilePluginSettingsService.....	19
8.5 EnterpriseConteXService.....	19
9. The User Interface (Views).....	20
10. Getting Started as a Developer	21
10.1 Suggested reading order.....	21
10.2 Mental models to keep.....	21

10.3 Common gotchas	21
Appendix A: The IPillarStorage API.....	22
Appendix B: The Data Model.....	23

1. Introduction

1.1 What this software does

SnapStak Mobile is a .NET MAUI Android application that deconstructs other mobile apps. Specifically, it targets the very large class of Android apps that are built as a thin native shell around a web view - apps made with Capacitor, Cordova, Ionic, Median, WebToApp and similar frameworks. SnapStak Mobile finds those apps on the device, opens their web layer inside a controlled WebView, captures exactly what the browser renders, and turns it into production-ready source code or design-tool files - all on the phone itself, with no server required in the default build.

In short: the Desktop product captures pages from a desktop browser through a Chrome extension. This product does the equivalent for installed mobile WebView apps, where the "browser" is an Android WebView the app embeds. The capture target is different and the platform is different, but the underlying idea - ConteX Law - is the same.

1.2 Why it is built this way: ConteX Law

Like the Desktop engine, SnapStak Mobile is an application of ConteX Law: the principle that a generative model stops fabricating when its input is specified completely. The app measures a captured screen into four complete, non-overlapping descriptions - the four pillars - and hands the model a specification with no gaps left to fill. The model transcribes instead of inventing.

Pillar	Question it answers	On mobile, it contains
Structure	What the thing IS	Exact geometry, hierarchy, colours and typography from the WebView, serialised as an SVG treated as immutable truth.
Behaviour	What it DOES	The captured CSS (and any JS) as source data for description.
Influence	What SHAPED it	The Android WebView environment: Chromium engine, mobile viewport, device pixel ratio 3.0, light scheme.
Objective	What it must BECOME	The target framework the user picks, fixed to a mobile device profile and the captured viewport width.

1.3 How Mobile differs from the Desktop engine

This codebase began as a port of the Desktop CON10X Web Domain engine and was then adapted for mobile. The original Desktop build relied on a remote CON10X server; that dependency has been removed for the default build. Understanding what changed is the fastest way to understand the app.

Concern	Desktop	SnapStak Mobile
Capture target	Any web page in Chrome	The web layer of an installed Android WebView app
Capture mechanism	Chrome extension (background + content scripts)	A custom Android WebView handler that intercepts requests and injects the engine
Engine source	content.js served from the local server	content.mobile.js bundled as an embedded app asset (LocalEngineService)
Pipeline	Runs in the Blazor WASM client / local server	Runs fully on-device (MobileConteXPipelineService), no server in the default build

Concern	Desktop	SnapStak Mobile
Storage	Browser IndexedDB	FileSystem.AppDataDirectory (FilePillarStorage)
Output	Project zip download	Plugin files shared via the Android share sheet, plus optional code generation

The shared core. Crucially, the actual CON10X engine - StructureService (DOM to SVG tree), SvgSerializer, the translator plugin contract and the data models - is the same code as the Desktop, carried in this project under Engine/Shared. The mobile-specific work is all about capture (getting a faithful DOM out of an embedded WebView) and platform plumbing (MAUI, Android, on-device storage). When you read the engine, you are reading Desktop code; when you read the capture layer, you are reading genuinely mobile-specific code.

1.4 Who this document is for

This is for a developer who has just been handed the repository. It assumes you can read C# and a little MAUI/XAML and Android, but nothing about this project. It starts with opening and building the solution in Visual Studio 2022, then explains the architecture, then walks every part in detail, and ends with reference appendices.

2. The Visual Studio 2022 Solution

The repository is a single-project Visual Studio 2022 solution targeting **.NET 9 for Android**. Open `SnapStakMobile.sln` in Visual Studio 2022 (17.14 or later) and it loads as one MAUI project, `SnapStakMobile`.

2.1 Prerequisites

- **Visual Studio 2022** (17.14 or later) with the ".NET Multi-platform App UI development" (MAUI) workload installed. The solution header declares Visual Studio Version 17.14.
- **.NET 9 SDK with the Android workload**. The project targets `net9.0-android` and sets a minimum Android of API 26 (Android 8.0).
- **An Android device or emulator running API 26+**. A physical device is recommended because the app scans installed apps, which is more meaningful on a real phone.
- **The Android SDK and a JDK** (both installed by the MAUI workload). To deploy to a physical device you will also need USB debugging enabled on the phone.

2.2 Solution structure

Unlike the Desktop solution (two projects), this is one MAUI project. Its folders map cleanly onto responsibilities.

Folder	Contents
(root)	App bootstrap: <code>App.xaml(.cs)</code> , <code>AppShell.xaml</code> , <code>MauiProgram.cs</code> (DI), <code>AppConfig.cs</code> , <code>appsettings.json</code> .
<code>Platforms/Android/</code>	Android-specific code, most importantly <code>LocalAssetWebViewHandler.cs</code> (the capture engine) and <code>MainActivity / MainApplication / the manifest</code> .
<code>Engine/Shared/</code>	The ported Desktop CON10X engine: <code>StructureService</code> , <code>SvgSerializer</code> , <code>SvgDeserializer</code> , the plugin contract, and all data models. Compiled unchanged.
<code>Engine/Plugins/</code>	Mobile-specific plugins (the SnapStak SVG plugin).
<code>Pipeline/</code>	<code>MobileConteXPipelineService</code> - the on-device orchestrator.
<code>Services/</code>	App scanning, asset extraction, the local engine, pairing, plugin settings, the database, the enterprise client.
<code>Storage/</code>	<code>FilePillarStorage</code> - the on-device <code>IPillarStorage</code> implementation.
<code>Views/</code>	The MAUI pages (XAML + code-behind): <code>Splash</code> , <code>AppList</code> , <code>Scan</code> , <code>Deconstruct</code> , <code>Transform</code> , <code>Settings</code> , <code>SavedApps</code> , <code>Pairing</code> .
<code>Models/</code>	App and signature model types (<code>AppItem</code> , <code>SavedApp</code> , <code>ScannedApp</code> , <code>SignatureDefinition</code>).
<code>Resources/</code>	Fonts, images, styles, and <code>Raw/</code> which holds <code>content.mobile.js</code> and <code>signatures.json</code> .

2.3 The shared engine and how it is linked

The `Engine/Shared/` folder is the Desktop CON10X engine carried into this project. As the migration notes explain, those files compile unchanged in the MAUI context - the C# is platform-neutral. In the repository they are present as source files in the project so the app builds standalone. Conceptually,

this is the same code that lives in the Desktop's SnapStak.Wasm.Client project under the same namespaces (`SnapStak.Wasm.Client.Engine.*` and `SnapStak.Wasm.Client.Models.*`), which is why the mobile pipeline and plugins reference those namespaces directly.

Why it matters to a new developer: when you open a file under Engine/Shared and see Desktop namespaces, that is expected. Do not change those files to "fix" the namespace - they are intentionally the shared engine. Mobile-specific behaviour belongs in Platforms, Pipeline, Services, Storage and Views.

2.4 NuGet packages

Add or update packages only through the Visual Studio 2022 NuGet Package Manager UI (right-click the project, Manage NuGet Packages, Browse, search, Install). Do not hand-edit the .csproj.

Package	Version	Why it is here
Microsoft.Maui.Controls	9.0.10	The MAUI UI framework.
Microsoft.Maui.Controls.Compatibility	9.0.10	Compatibility layer for older control patterns.
Microsoft.Maui.Essentials	9.0.10	Device APIs: Preferences, SecureStorage, FileSystem, Share, Launcher.
Xamarin.AndroidX.WebKit	1.15.0.2	The AndroidX WebView APIs, including WebViewAssetLoader used to serve extracted app files.
sqlite-net-pcl	1.9.172	The saved-apps and scan-history database.
SQLitePCLRaw.bundle_green	2.1.11	Native SQLite provider for sqlite-net.
SauceControl.Blake2Fast	2.*	BLAKE2b hashing required by the Penpot translator plugin.
Newtonsoft.Json	13.*	JSON across the pipeline and settings services.

Note the [NU1608](#) warning is suppressed in the project: Xamarin.AndroidX.WebKit pulls in newer transitive AndroidX packages than some binding packages declare as their upper bound. The resolved versions are higher but compatible - a known and expected pattern with AndroidX bindings in MAUI.

2.5 Build configurations: Standalone vs Enterprise

This is the most important build fact in the project. The app compiles in two distinct variants, selected by configuration and a compile-time symbol:

Variant	Symbol	What runs where
Standalone	(default; Debug / Release)	The full CON10X engine runs on-device. Extraction, Structure, plugins all local. No server. Subscription via Paystack.
Enterprise	ENTERPRISE	Extraction still runs on-device, but Structure + Behaviour + Constructor run on the configured Enterprise CON10X server, reached over HTTP + SSE.

Throughout the codebase you will see `#if ENTERPRISE ... #else ... #endif` blocks. MauiProgram registers either `EnterpriseContextService` or the local `MobileContextPipelineService` and `TranslatorPluginHost`; AppConfig exposes either the enterprise server endpoints or the Paystack subscription endpoints; and DeconstructPage / TransformPage branch between posting to the server and running the local pipeline. When reading any of those files, first note which variant the block belongs to.

2.6 Running it on a device or emulator

1. **Select the SnapStakMobile project and an Android target** (a connected device or a running emulator) in the Visual Studio toolbar.
2. **Choose the Debug configuration** (Standalone) for normal development. Press F5 to build, deploy and launch.
3. **Grant the app the "query all packages" capability** if prompted - the scanner needs to enumerate installed apps to find WebView apps.
4. **On first launch the app loads its bundled signatures and engine asset**, then shows the app list. Run a scan to detect WebView apps, tap one to deconstruct it, then choose a framework or export a plugin file.

For the Enterprise variant, add the ENTERPRISE compile symbol to the build configuration and set the enterprise server URL and API key in Settings (stored in SecureStorage). The app will then post extractions to that server instead of processing locally.

3. Architecture Overview

3.1 The end-to-end flow

Follow one app all the way through once. Every file you read later slots into one of these steps.

5. **Scan.** AppScannerService enumerates installed apps and inspects each APK to decide whether it is a WebView app and, if so, which framework and what its entry URL or local index file is.
6. **Prepare.** For a local-asset app, AssetExtractorService unpacks the app's web files (everything under assets/) into a private cache folder so they can be served to the WebView. For a remote-URL app, the URL is used directly.
7. **Open.** DeconstructPage hosts a MAUI WebView wired to the custom LocalAssetWebViewHandler. The handler intercepts every request: it serves local files, stubs out framework JavaScript, strips CSP, and injects content.mobile.js into the page.
8. **Capture.** When the user taps Deconstruct, the page calls extractMobile() inside the WebView via evaluateJavaScript, then polls a global for the JSON result. content.mobile.js walks the rendered DOM with getBoundingClientRect() and getComputedStyle() and returns the Structure geometry plus CSS.
9. **Build pillars.** MobileConteXPipelineService parses that JSON into a TransformRequest, builds the Structure SVG with the shared StructureService and SvgSerializer, synthesises the Influence and Objective pillars for the mobile context, and writes everything to on-device storage.
10. **Translate.** Each enabled translator plugin (Penpot, Figma, SnapStak SVG, Canva) runs against the same SVG tree, producing one export file each.
11. **Generate or export.** On TransformPage the user either generates framework code (through OpenRouter / the subscription server) or shares the plugin export files via the Android share sheet.

3.2 The four pillars on mobile

The pillars are the same concept as Desktop, but two of them are synthesised rather than measured, because a single embedded WebView capture has a fixed environment. In `ParseExtractionJson`, Influence is built as a fixed Android WebView profile (Chromium engine, the captured viewport, device pixel ratio 3.0, light colour scheme) and Objective is built as a mobile target at the captured width. Structure and Behaviour come from the live capture. This is a deliberate simplification: on a phone there is one device, one viewport, one environment, so those pillars are constants rather than choices.

3.3 Where the data lives

Store	Location	Holds
App private files	<code>FileSystem.AppDataDirectory/snapstak/{uuid}/{componentId}/</code>	All pillar files and plugin outputs for each deconstructed component (FilePillarStorage).
App cache	<code>FileSystem.CacheDirectory/snapstak_assets/</code>	Extracted web assets per app

Store	Location	Holds
		version (AssetExtractorService). Android may evict under storage pressure.
SQLite database	App data directory	Saved apps and scan history (AppDatabaseService).
Preferences	Android SharedPreferences	Plugin toggles and similar settings (MobilePluginSettingsService).
SecureStorage	Android Keystore-backed	Device UUID, pairing token, OpenRouter / enterprise API key, enterprise server URL.
Embedded assets	Inside the APK	content.mobile.js (the engine) and signatures.json (the scanner's framework definitions).

4. App Discovery: Finding WebView Apps

Before anything can be captured, SnapStak Mobile has to find apps worth capturing. This is mobile-specific work with no Desktop equivalent, and it is genuinely clever, so it is worth understanding in detail.

4.1 AppScannerService - the two-layer APK scan

Location: `Services/AppScannerService.cs`. Given the list of installed non-system apps, the scanner decides which are WebView apps. Scanning every APK's bytecode fully would be far too slow, so it uses a two-layer approach with an incremental-scan cache.

12. **Skip unchanged apps.** Each app's APK last-modified time is compared against the last scan. If nothing changed, the app is skipped entirely - so re-scans are fast.
13. **Layer 1 - native triage (2-5 ms).** `CanBeWebViewApp` opens the APK as a ZIP and reads only the central directory (the file list), never the bytecode. If none of the known WebView config files (for example `assets/capacitor.config.json`) are present, the app is definitively native and is skipped before the expensive scan.
14. **Layer 2 - DEX class-marker scan.** `InspectApk` streams every `classesN.dex` file in fixed-size chunks (with overlap so markers spanning a chunk boundary are not missed) and records which framework class markers appear. It scans all DEX files before deciding, then picks the most specific matching signature - the one with the most class markers all present.

A real subtlety worth calling out: Capacitor apps embed the Cordova bridge for compatibility, so a naive scan that stopped at the first match could misclassify a Capacitor app as Cordova. The scanner avoids this by accumulating every marker across all DEX files first, then choosing the most specific signature, so `com.getcapacitor` wins over `org.apache.cordova`. There is also a fallback for R8-obfuscated release builds where the Capacitor class name is stripped but `capacitor.config.json` is still present.

Confirmation and URL extraction. A class-marker match is then confirmed by checking that the framework's config file actually exists in the APK (this eliminates native apps that merely embed a framework library). The scanner then extracts the app's entry point - either a remote URL from the config (for example `server.url`) or a local index file path (for example `assets/public/index.html`). Only apps with a usable URL or local index are surfaced to the user.

4.2 Signatures and SignatureCacheService

The scanner is entirely data-driven. The framework definitions live in `Resources/Raw/signatures.json` and are loaded by `SignatureCacheService`. Nothing about the frameworks is hardcoded in the scanner - it works off this list. Each signature defines the framework name, a colour badge, the DEX class markers, the confirming config file, the keys to read the URL from, and the candidate local index paths.

Signature field	Purpose
<code>framework</code>	Display name and badge (Capacitor, Cordova, Median, WebToApp, ...).
<code>classMarkers</code>	DEX strings that identify the framework (e.g. <code>com.getcapacitor</code>).
<code>file</code>	A config file whose presence confirms the framework (e.g.

Signature field	Purpose
	assets/capacitor.config.json).
urlKeys	Dotted keys to read the entry URL from the config (e.g. server.url).
localIndexPaths	Candidate local index files for offline apps (e.g. assets/public/index.html).
appIdPrefix / appIdLabels	Refinement by application id - e.g. an app.lovable.* id is badged as Lovable, app.base44.* as Base44.

Why this is a good design: supporting a new WebView framework is a data change to signatures.json, not a code change. In the Standalone build the signatures are bundled in the app; the service also supports fetching them, which would let the framework matrix be updated without shipping a new build.

4.3 AssetExtractorService - unpacking local apps

Location: `Services/AssetExtractorService.cs`. For an app whose UI is bundled inside the APK (rather than loaded from a remote URL), its web files must be made available to the WebView. This service extracts everything under the APK's top-level `assets/` folder into a private cache directory and returns a file path to the index.

Caching that invalidates correctly. The cache key combines the package name, the APK's last-modified ticks, and an extractor version constant. That means the cache survives restarts, automatically invalidates when the app is updated (timestamp changes) or when the extractor logic changes (version bump), and can be evicted by Android under storage pressure. It extracts all of `assets/` rather than just the index's folder, because apps routinely reference sibling asset directories for CSS and fonts.

5. The Capture Layer (Android WebView)

This is the technical heart of SnapStak Mobile and where the hardest problems are solved. The job sounds simple - load an app's web UI and run a script in it - but doing so reliably across CSP, framework bundles, and native bridges is the bulk of the engineering. It all lives in `Platforms/Android/LocalAssetWebViewHandler.cs`.

5.1 LocalAssetWebViewHandler - the heart of capture

This is a custom MAUI `WebViewHandler` that replaces the default for the app's `WebView` (wired up in `MauiProgram`). It controls the native Android `android.webkit.WebView` directly so it can intercept every network request and inject script in ways the cross-platform MAUI `WebView` cannot. It contains several cooperating classes:

Class	Role
<code>LocalAssetWebViewHandler</code>	The handler. Installs the right <code>WebViewClient</code> and manages the page-load lifecycle, including a documented workaround for MAUI's <code>SetVirtualView</code> ordering.
<code>LocalAssetWebViewClient</code>	For local-asset apps. Serves files from the extracted cache, stubs framework JS, injects native-bridge stubs and the engine.
<code>SnapStakWebViewClient</code>	For remote-URL apps. Fetches the HTML itself, strips the CSP header, and injects the engine before returning the response.
<code>SnapStakJsInterface</code>	A JavaScript interface exposing the engine script to the page (<code>getScript</code>) so it can be bootstrapped into new page contexts.
<code>CachePathHandler</code>	An AndroidX <code>WebViewAssetLoader</code> path handler that maps virtual asset URLs to files on disk with correct MIME types.

The lifecycle workaround. MAUI's handler runs all its property mappers and then issues a navigation to `about:blank` for a null `Source`. The handler documents this exactly and works with it: it installs the correct `WebViewClient` during the mapper pass and again permanently in `ConnectHandler`, then overrides the `about:blank` navigation in `SetVirtualView` by calling `LoadUrl` with the real asset URL. This is the kind of detail that looks mysterious until you read the comments - which are thorough and worth trusting.

5.2 Local-asset apps vs remote-URL apps

The two `WebViewClient` subclasses exist because the two kinds of `WebView` app need different handling, even though both end with the engine injected into the page.

- **Local-asset apps** (UI bundled in the APK) use a virtual domain, `appassets.androidplatform.net`, served by the `WebViewAssetLoader` and `CachePathHandler` from the extracted cache. The client owns this domain entirely and never lets it reach the network.
- **Remote-URL apps** (UI loaded from a live server) are handled by fetching the main HTML document directly with an `HttpClient` (decompressing `gzip/brotli` first), stripping CSP, injecting the engine as a script tag, and returning the modified response.

5.3 Defeating CSP and framework JavaScript

Two things normally stop a script from being injected and running inside someone else's app:

- **Content Security Policy.** CSP blocks inline scripts and eval(). The handler removes it at the source - it strips the Content-Security-Policy and Content-Security-Policy-Report-Only response headers for remote apps, and strips CSP meta tags from the HTML for local apps. With CSP gone, the injected engine executes freely. The comments are explicit that stripping CSP from the response is the only reliable method, because every in-page injection route is otherwise blocked.
- **Framework JavaScript.** The app only needs HTML and CSS to render a capturable UI. The framework's own JavaScript (React/Vue/Ionic bundles, polyfills, runtime loaders, native bridges) tends to make network calls, attempt auth redirects, or crash when native APIs are absent - any of which can stop the DOM from reaching a usable state. So LocalAssetWebViewClient stubs framework and vendor bundles with an empty-but-valid JS response (not a 404, which would cause module-not-found errors), while still allowing the app's own page JS to run.

The Capacitor / Cordova patch. WebView frameworks expect a native bridge that is not present here. The handler injects small Capacitor and Cordova stubs so that calls into the bridge resolve harmlessly (every plugin call returns a resolved promise, and the Cordova deviceready event is fired) instead of throwing. It also patches vendor bundles at serve time, replacing Capacitor's "Not implemented on web" throw with a silent resolved promise, so the app's async startup chain completes rather than crashing.

5.4 Injecting the engine and the native bridge stubs

The engine (content.mobile.js) is injected directly into the HTML byte stream as the first script in the head, alongside the Capacitor and Cordova stubs. Injecting into the stream means the browser runs it as part of the page itself - same origin, no CSP, no timing race. The engine script is read from `App.Engine.Read()` (the LocalEngineService cache) with a fallback to a pending-script static, which removes the race where the request interception fires before the page navigation has set the script.

5.5 content.mobile.js - the extractor

Location: `Resources/Raw/engine/content.mobile.js` (about 115 KB). This is the mobile-trimmed version of the Desktop `content.js`: the Chrome-extension messaging, the component selector, and the multi-pass page logic are removed, leaving a clean WebView extractor. Its entry point is `extractMobile()`, which the page calls and which returns the capture as a JSON object.

What it does, in order: it scrolls the page to trigger lazy-loading and IntersectionObserver content, waits for images and layout to settle, prefetches inline SVG sprite definitions, stamps segment ids, expands horizontal carousels so off-screen items are measured, and then serialises the DOM. For each element it records the tag, text, classes, a rect from `getBoundingClientRect()`, and the computed style from `getComputedStyle()` - exactly the raw material the shared StructureService needs to build the SVG. The result is returned as visible elements plus page metadata (URL, title, viewport).

Why it runs in the page. As on Desktop, `getComputedStyle()` only returns the real, fully-resolved values inside the page's own JavaScript context. That is why the whole capture layer exists: to get this one script running inside the target app's WebView with a usable DOM.

6. The On-Device Pipeline

Once the capture JSON is in hand, the pipeline turns it into the four pillars and the export files - all on the device in the default build. This is the part most directly inherited from Desktop.

6.1 DeconstructPage - orchestrating extraction

Location: `Views/DeconstructPage.xaml.cs`. This page hosts the WebView and drives extraction. It is created through an async factory, `CreateAsync`, which first ensures the engine is loaded (`LocalEngineService`) and stashes it for the handler, then wires up either the local pipeline or the enterprise service depending on the build.

The extraction handshake. When the user taps Deconstruct, the page generates a componentId, injects a tiny chrome.runtime shim (some app code expects it to exist), then calls `extractMobile()` via `evaluateJavaScript` and stores the result on a page global. It polls that global every 500 ms (up to a 2-minute timeout), and once the JSON arrives it forces a garbage collection (the snapshot can be large) and hands the JSON to the pipeline. A timeout is surfaced as a likely CSP block - the most common failure mode.

6.2 MobileConteXPipelineService - building the pillars

Location: `Pipeline/MobileConteXPipelineService.cs`. This is the on-device replacement for the Desktop server round-trip. Its `TransformAsync` method runs the whole local pipeline:

15. **Parse.** `ParseExtractionJson` turns the WebView's JSON into a `TransformRequest`. It is tolerant of several shapes (mainSnapshot, domSnapshot, or a root visible[] array), and it synthesises the Influence and Objective pillars for the Android WebView context.
16. **Build Structure.** Using the shared `StructureService` it builds the SVG node tree, applies computed styles, then serialises with `SvgSerializer` to the master SVG - the Structure pillar - and writes it to storage.
17. **Write Behaviour, Influence, Objective.** The captured CSS (and any page-map CSS) is written as the Behaviour source; the synthesised Influence and Objective pillars are written too. The component is registered in the on-device session manifest.
18. **Run plugins.** Enabled translator plugins run against the tree, each producing one export file (next section).

It returns a result carrying the componentId, the element count, the SVG dimensions, and the list of plugin output files with their paths and sizes - which `TransformPage` then turns into share buttons.

6.3 The translator plugins

The plugin system is the same two-phase contract as Desktop (`IConteXTranslatorPlugin`: `DeclareResources` then `Translate`), discovered through `TranslatorPluginHost`. The mobile pipeline filters to only the plugins the user has enabled, fetches any remote resources they declare in parallel, then runs each one. Failures are isolated - a broken plugin is logged and skipped.

Plugin	Output	Notes
Penpot	<code>.penpot</code>	Binary archive. Requires BLAKE2b hashing (<code>SauceControl.Blake2Fast</code>). Enabled by default.

Plugin	Output	Notes
Figma	.figma.svg	Annotated SVG for the Figma importer. Enabled by default.
SnapStak SVG	.snapstak.svg	The canonical CON10X Structure SVG. Mobile-only toggle. Enabled by default but never shared to the user (it is core IP) - see 7.3.
Canva	.canva.pdf	PDF for Canva Connect. Disabled by default (needs Canva credentials).

6.4 FilePillarStorage - on-device storage

Location: `Storage/FilePillarStorage.cs`. This is the mobile `IPillarStorage` implementation. Where the Desktop client uses browser IndexedDB, this writes plain files under `FileSystem.AppDataDirectory/snapstak/{userId}/{componentId}/`. There is no encryption layer: the Android per-app sandbox is the security boundary - the directory is private to the app and removed on uninstall. The same interface shape as Desktop means the shared engine code reads and writes pillars without knowing it is on a phone. The full API is in Appendix A.

7. Code Generation and Export

7.1 TransformPage - framework choice and generation

Location: `Views/TransformPage.xaml.cs`. After a successful deconstruct, this page lets the user pick a target category (web or native) and a framework, then either generate code or export the plugin files. The framework cards and selection logic are identical across both builds; only the generation path differs.

7.2 The Standalone and Enterprise generation paths

- **Standalone** reads the user's OpenRouter API key (obtained through the pairing token, not typed in), checks the OpenRouter credit balance first (warning the user if it is empty or low and offering to open the top-up page), and then drives generation. Code generation uses the same ConteX Law approach as Desktop - the four pillars assembled into one prompt at a low temperature - producing the component code the user downloads or shares.
- **Enterprise** posts the component to the configured server's process-session endpoint and listens on an SSE stream for `COMPONENT_COMPLETE` / `SESSION_COMPLETE` / error events, then downloads the generated package with the returned token. This mirrors the Desktop server pipeline exactly, with the phone as a thin client.

7.3 Exporting plugin outputs via the share sheet

This is the mobile-native equivalent of the Desktop zip download. After deconstruction, the plugin output files appear as share buttons; tapping one opens the Android share sheet so the file can go to Penpot, Figma, Canva, cloud storage, or any app the user chooses. One important rule is enforced here: the `snapstak-svg` output is filtered out of the visible list. The SnapStak SVG is the internal CON10X Structure pillar - core IP - so it is generated and stored on-device for the pipeline but never exposed to the user through the share sheet.

8. Services and Supporting Systems

8.1 LocalEngineService

Loads `content.mobile.js` from the app's embedded assets once and caches it in memory for the session. It is never written to disk and never sent over the network, and it is cleared from memory when the app suspends (to avoid leaving the engine in a memory dump). This is the mobile replacement for the Desktop's server-served engine.

8.2 PairingService and subscription

Location: `Services/PairingService.cs`. Handles the Desktop-to-Mobile pairing and subscription. A device UUID is generated once and stored in `SecureStorage`; it is the device identity and is used as the key material for encrypting generated files (so only the paired Desktop can decrypt them, and a factory reset permanently revokes access). Pairing works over the local network: the Desktop shows a 6-digit PIN and advertises itself via mDNS, the phone scans, connects, and exchanges an encrypted token. Subscription validity is then checked against the subscription server. In the Standalone build, the OpenRouter key used for code generation is delivered through this pairing token rather than typed in by the user.

8.3 AppDatabaseService

A small SQLite layer (sqlite-net) storing two tables: saved apps (the user's chosen apps, with icon and metadata) and scan history (package name plus APK timestamp, which powers the scanner's skip-unchanged optimisation). It exposes simple async CRUD methods.

8.4 MobilePluginSettingsService

Stores the translator-plugin on/off toggles in MAUI Preferences (backed by Android `SharedPreferences`), mirroring the Desktop plugin settings which use browser `localStorage`. Mobile differs in having no output-path fields (exports go to the share sheet), no Framer integration, and an added SnapStak SVG toggle. Defaults: Penpot, Figma and SnapStak SVG enabled; Canva disabled.

8.5 EnterpriseConteXService

Location: `Services/EnterpriseConteXService.cs`. Present only in the Enterprise build. It is the client for the remote CON10X server: it posts the extracted DOM to the transform endpoint, fires process-session, listens to the SSE progress stream, and downloads the generated package. Credentials (the API key and server URL) come from `SecureStorage` and are sent as bearer auth plus a set of X-ConteX headers.

9. The User Interface (Views)

The UI is a MAUI Shell app with two tabs (Apps and Settings) defined in `AppShell1.xaml`. Navigation flows from the app list into deconstruction and transformation. Each page is a XAML file plus a code-behind.

Page	Role
SplashPage	Launch screen; initial load and routing.
AppListPage	The home screen: the user's saved / detected WebView apps.
ScanPage	Runs AppScannerService with live progress (inspected / skipped / triaged / found) and surfaces detected apps.
DeconstructPage	Hosts the WebView, injects the engine, runs extraction, and invokes the pipeline.
TransformPage	Framework selection, code generation, and plugin export share buttons.
SettingsPage	API key / subscription, plugin toggles, and the danger-zone data clear.
SavedAppsPage	Manage the saved-apps list.
PairingPage	Enter the Desktop PIN to pair the device.

10. Getting Started as a Developer

10.1 Suggested reading order

Read the code in this order rather than alphabetically:

19. `MauiProgram.cs + AppConfig.cs` - the DI wiring and the Standalone/Enterprise split.
20. `AppScannerService.cs` - how WebView apps are found (the most mobile-specific logic).
21. `Platforms/Android/LocalAssetWebViewHandler.cs` - the capture engine; read the comments carefully.
22. `Views/DeconstructPage.xaml.cs` - how extraction is triggered and polled.
23. `Pipeline/MobileConteXPipelineService.cs` - how the capture becomes the four pillars.
24. `Views/TransformPage.xaml.cs` - generation and export.
25. The `MIGRATION.md` in the repo - it summarises exactly what changed from the Desktop engine.

10.2 Mental models to keep

- **Capture is the hard part, not the engine.** The CON10X engine is shared, proven Desktop code. The mobile value and the mobile difficulty are in getting a faithful DOM out of someone else's embedded WebView - the scanner and the WebView handler.
- **Two of the four pillars are synthesised.** On a phone there is one device and one viewport, so Influence and Objective are fixed constants built in `ParseExtractionJson`, not user choices.
- **Always know which build you are in.** Standalone runs everything on-device; Enterprise offloads everything after extraction to a server. The `#if ENTERPRISE` blocks change behaviour throughout.
- **CSP and framework JS are the enemies of capture.** Most capture failures trace back to one of these. The handler strips CSP and stubs framework JS precisely so the DOM can reach a capturable state.

10.3 Common gotchas

- **"Extraction timed out"** - usually CSP blocking the injected engine, or framework JS preventing the DOM from settling. Check the WebView handler's stubbing and CSP stripping for that app type.
- **An app is not detected by the scan** - its framework may lack a signature in `signatures.json`, or it may have no extractable URL / local index. Fileless frameworks can only be found by the DEX scan.
- **Stale extracted assets** - bump the extractor version constant in `AssetExtractorService` to force re-extraction when the set of extracted files changes.
- **Engine not injected** - `LocalEngineService` must have loaded `content.mobile.js` before the page loads; `CreateAsync` ensures this, and the handler falls back to `App.Engine.Read()`.
- **NuGet** - always via the Visual Studio 2022 Package Manager UI; never hand-edit the `.csproj`.

Appendix A: The IPillarStorage API

FilePillarStorage implements the shared `IPillarStorage` interface against the Android file system. Knowing its shape tells you exactly what is persisted per component and how it is named (files under `AppDataDirectory/snapstak/{uuid}/{componentId}/`).

Group	Methods
Path resolution	ResolveComponentDir, ResolveIconsDir, ResolveSessionRoot
Structure	WriteSvg, ReadSvg, WriteViewportSvg
Behaviour	WriteCssMd, WriteJsMd, ReadBehaviour, BehaviourMdExists, WriteCssJson, WriteJsJson, ReadCssJson
Influence / Objective	WriteInfluence, ReadInfluence, WriteObjective, ReadObjective
Hidden content	WriteHiddenElements, WriteHiddenComponents (+ SVG variants), ReadHiddenElements
Source HTML / icons	WriteSourceHtml, ReadSourceHtml, WriteIcon, ReadIcon
Plugin output	WriteTranslatorOutput (one file per plugin, by file extension)
Session manifest	RegisterComponentInManifest, ReadSessionManifest, UpdateComponentStatus, UpdateSessionStatus, CleanupSessionFiles, CleanupPillarFiles
Snapshot	WriteSnapshot, ReadSnapshot, ReadSnapshotDirect, ClearSnapshot

Appendix B: The Data Model

B.1 Mobile model types (Models/)

Type	Role
ApplItem	A detected app in the UI: name, package, APK path, framework badge and colour, extracted URL, local-asset flag and index path, icon.
SavedApp	A persisted app record (SQLite): name, package, URL, framework, icon (base64), saved timestamp, APK path and local index.
ScannedApp	A scan-history record: package name, APK last-modified, scanned-at - used to skip unchanged APKs.
SignatureDefinition / AppldLabel	A WebView framework definition (class markers, config file, URL keys, local index paths) and an application-id-based label refinement.

B.2 Shared engine model types (Engine/Shared/Models/)

These are the same types as the Desktop engine, carried in this project. They are the shapes that flow from the capture into the pillars.

Type	Role
DomElement / DomSnapshot / PageMapEntry	The captured DOM: per-element geometry and computed style, the element list with page dimensions, and the per-section page map.
InfluenceData	The environment pillar - on mobile, the synthesised Android WebView profile.
ObjectiveData	The target pillar - on mobile, a fixed mobile target at the captured width.
TransformRequest	The pipeline input assembled from the extraction JSON, carrying all four pillars and the client type (maui).
SessionManifest / SessionComponent / PillarStatus	The on-device project record and per-component pillar readiness.

This document describes the SnapStak Mobile application and its on-device CON10X pipeline. The source is at github.com/SnapStak-AI/SnapStak-Mobile and is licensed under AGPL-3.0. Contributions and derivative works are welcome under the terms of that licence.